SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Machine Learning

ASSESSMENT TOOL 1 – Analytical Problem Solving (5 QUESTIONS)

Course Code:	MLA0407	**Course Title:**	**Deep Learning**
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – **Analytical problem solving**
Weightage:	35% of CIA	**Total Marks:**	(5 Questions × 10 Marks)
CO1 Statement:	***Apply the fundamental concepts of machine learning and neural networks to design and analyze learning algorithms using perceptrons, multilayer perceptrons, forward propagation, backpropagation, gradient descent optimization techniques, and Maximum Likelihood Estimation for solving real-world problems.***		

Student Name: T Bhupal Reg.no. 192425243

Section/Batch: 2024 Date: 17-07-2026

Code of Conduct

I T Bhupal/192425243 (NAME/ REG NO) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: bhupal

Q1. Design a machine learning solution for predicting customer loan approval using a suitable learning algorithm. Describe the major stages involved in model development, including data preparation, model training, and performance evaluation. (10 marks)

To predict whether a customer will be approved for a loan, a Random Forest Classifier is a highly suitable algorithm. It handles non-linear relationships well, is robust against overfitting, and provides feature importance (e.g., highlighting that credit score matters more than age).

The model development process involves three major stages:

1. Data Preparation

• Data Cleaning: Handle missing values (e.g., imputing missing income with the median) and remove duplicates or outliers that could skew the model.

• Feature Engineering: Create new variables that indicate creditworthiness, such as Debt-to-Income (DTI) ratio.

• Encoding: Convert categorical data (like 'Employment Status' or 'Marital Status') into numerical formats using techniques like One-Hot Encoding.

• Scaling: Standardize numerical features (like Income and Loan Amount) so that large values do not disproportionately influence the algorithm.

2. Model Training

• Data Splitting: Divide the dataset into a training set (typically 80%) to teach the model, and a testing set (20%) to evaluate it.

• Algorithm Application: Feed the training data into the Random Forest algorithm, which builds multiple decision trees based on random subsets of the data and features.

• Hyperparameter Tuning: Adjust the model's settings (like the number of trees or the maximum depth of each tree) using cross-validation to find the optimal configuration.

3. Performance Evaluation

• Precision and Recall: For loan approvals, minimizing False Positives (approving bad loans) is critical. Precision measures how many approved loans were actually good. Recall measures how many good loans were successfully identified.

• F1-Score: The harmonic mean of precision and recall, providing a balanced metric.

• ROC-AUC: Measures the model's ability to distinguish between the two classes (approved vs. rejected) across different threshold levels.

Q2. Describe the architecture of an Artificial Neural Network (ANN) consisting of input, hidden, and output layers. Illustrate the role of artificial neurons, weights, bias, and activation functions in processing input data to generate predictions. (10 marks)

An Artificial Neural Network is a computational model inspired by the human brain, structured in layers of interconnected nodes (neurons).

Layer Architecture

• Input Layer: Receives the raw data (e.g., age, income, credit score). It does not perform computations; it simply passes the data to the next layer. The number of nodes equals the number of features.

• Hidden Layers: One or more layers situated between the input and output. These layers perform the heavy lifting, extracting patterns and complex relationships from the data.

• Output Layer: The final layer that produces the prediction. For binary classification (e.g., yes/no), it typically contains a single node.

Core Processing Components

• Artificial Neurons (Nodes): The basic unit of computation. Each neuron receives inputs, processes them, and passes an output forward.

• Weights: Numerical values assigned to the connections between neurons. They determine the importance or influence of a specific input on the output. Learning in an ANN is fundamentally the process of adjusting these weights.

• Bias: An additional constant added to the neuron's computation. It allows the activation function to shift left or right, ensuring the neuron can trigger even when all input values are zero.

• Activation Functions: Mathematical equations attached to each neuron that determine whether it should be activated. They introduce non-linearity into the network, allowing it to solve complex problems. Common examples include ReLU, Sigmoid, and Tanh.

Q3. Analyze the working of the Perceptron learning algorithm for solving a binary classification problem. Demonstrate the weight and bias update process for one iteration using a suitable numerical example. (10 marks)

The Perceptron is the simplest form of a neural network, used for binary classification. It takes multiple inputs, calculates a weighted sum, adds a bias, and passes the result through a step activation function to output a 1 or a 0.

If the prediction is wrong, the algorithm updates the weights and bias to reduce the error in the next pass.

The Update Rules:

• w_new = w_old + α * (y - y_hat) * x

• b_new = b_old + α * (y - y_hat)

(Where α is the learning rate, y is the actual target, y_hat is the predicted output, and x is the input).

Numerical Example

Assume a learning rate α = 0.1. We want to classify a data point where the actual target is y = 0.

• Inputs: x1 = 1, x2 = 0

• Initial Weights: w1 = 0.5, w2 = 0.5

• Initial Bias: b = 0.2

Step 1: Compute Weighted Sum (z)

z = (x1 * w1) + (x2 * w2) + b

z = (1 * 0.5) + (0 * 0.5) + 0.2 = 0.7

Step 2: Apply Step Activation Function

Assume the step function outputs 1 if z ≥ 0, and 0 otherwise.

Because 0.7 ≥ 0, the predicted output y_hat = 1.

Step 3: Calculate Error and Update

The prediction (1) does not match the target (0).

Error = y - y_hat = 0 - 1 = -1

• Update w1: 0.5 + 0.1 * (-1) * (1) = 0.4

• Update w2: 0.5 + 0.1 * (-1) * (0) = 0.5

• Update b: 0.2 + 0.1 * (-1) = 0.1

Q4. Illustrate the forward propagation process in a Multilayer Perceptron (MLP) using a simple neural network with one hidden layer. Compute the output values for the given inputs, weights, biases, and activation function. (10 marks)

Forward propagation is the process of pushing input data through the network layers to generate an output. Let's compute this for a simple network with 2 inputs, 1 hidden layer (with 2 neurons), and 1 output neuron.

Given Parameters:

- Inputs: $x1 = 0.5$, $x2 = 0.8$
- Hidden Layer Weights: $w11 = 0.2$, $w21 = 0.4$ (to neuron 1); $w12 = -0.5$, $w22 = 0.1$ (to neuron 2)
- Hidden Layer Biases: $b1 = 0.1$, $b2 = -0.2$
- Output Layer Weights: $wh1 = 0.6$, $wh2 = 0.8$
- Output Layer Bias: $bo = 0.3$
- Activation Function (all layers): ReLU ($f(x) = \max(0, x)$)

Step 1: Compute Hidden Layer Outputs

Hidden Neuron 1 (h1):

$zh1 = (x1 * w11) + (x2 * w21) + b1$

$zh1 = (0.5 * 0.2) + (0.8 * 0.4) + 0.1 = 0.1 + 0.32 + 0.1 = 0.52$

Activation: $ah1 = \max(0, 0.52) = 0.52$

Hidden Neuron 2 (h2):

$zh2 = (x1 * w12) + (x2 * w22) + b2$

$zh2 = (0.5 * -0.5) + (0.8 * 0.1) - 0.2 = -0.25 + 0.08 - 0.2 = -0.37$

Activation: $ah2 = \max(0, -0.37) = 0$

Step 2: Compute Final Output

Output Neuron (o1):

$zo1 = (ah1 * wh1) + (ah2 * wh2) + bo$

$zo1 = (0.52 * 0.6) + (0 * 0.8) + 0.3 = 0.312 + 0 + 0.3 = 0.612$

Final Activation: Output $= \max(0, 0.612) = 0.612$

Q5. Demonstrate the Backpropagation algorithm for training a neural network by calculating the output error, propagating the error through hidden layers, and updating the network weights using Gradient Descent. (10 marks)

Backpropagation is the core mechanism by which multi-layer neural networks learn. While forward propagation makes a prediction, backpropagation measures how wrong that prediction is and adjusts the network's weights backwards from the output to the input.

1. Calculating Output Error

Once forward propagation yields a prediction ($\hat{y}$), the network calculates the total loss (error) by comparing it to the actual target (y). A common metric for regression is Mean Squared Error (MSE), defined for a single instance as: $E = 1/2 * (y - \hat{y})^2$

2. Propagating Error Backward

The algorithm must determine how much each individual weight contributed to that total error. It does this using the Chain Rule from calculus to compute the gradient (partial derivative) of the error with respect to each weight ($\partial E/\partial w$). For a weight connected to the output layer, the chain rule breaks the derivative into three parts:

$$\partial E/\partial w = (\partial E/\partial \hat{y}) * (\partial \hat{y}/\partial z) * (\partial z/\partial w)$$

This tells the network the direction and magnitude to change the weight. For hidden layers, the error from the output layer is distributed backward based on the connection weights.

3. Updating Weights using Gradient Descent

Once the gradients are calculated for all weights, the network uses an optimization algorithm like Gradient Descent to update them. The weights are adjusted in the opposite direction of the gradient to minimize the error: $w_{new} = w_{old} - \alpha * (\partial E/\partial w)$ Here, α is the learning rate, which controls how large of a step the network takes during the update. This entire process (forward pass, calculate error, backward pass, update) repeats for many epochs until the model converges on accurate predictions.